

Fashion E-Commerce Website

Smoke Test Suite & Playwright TypeScript Automation Framework

Application: fashion-ecommerce-we-lunb.bolt.host

1. Smoke Test Suite

ID	Smoke Test	Priority	Type
SMK-001	Application loads successfully	P0	UI
SMK-002	Home page displays header/navigation	P0	UI
SMK-003	Product listing/category page opens	P0	Navigation
SMK-004	Products display name, price and image	P0	UI
SMK-005	User opens product details page	P0	Navigation
SMK-006	Product details show price and essential information	P0	UI
SMK-007	Product can be added to cart	P0	Cart
SMK-008	Cart opens successfully	P0	Cart
SMK-009	Cart contains selected product	P0	Cart
SMK-010	Cart quantity can be increased	P1	Cart
SMK-011	Product can be removed from cart	P1	Cart
SMK-012	Search field is available	P0	Search
SMK-013	Valid product search returns results	P0	Search
SMK-014	Login page opens	P0	Authentication
SMK-015	Valid user can log in	P0	Authentication
SMK-016	Logout works successfully	P1	Authentication
SMK-017	Checkout page opens for cart item	P0	Checkout
SMK-018	Required checkout fields are displayed	P0	Checkout
SMK-019	Payment/order submission can be initiated	P0	Checkout
SMK-020	Successful order confirmation is displayed	P0	Order

Recommended smoke flow: Launch → Home → Product Category → Product Details → Add to Cart → Cart → Checkout → Payment → Order Confirmation.

2. Recommended Playwright + TypeScript Project Structure

```
fashion-ecommerce-playwright/
├── tests/
│   ├── smoke/
│   │   ├── home.smoke.spec.ts
│   │   ├── product.smoke.spec.ts
│   │   ├── cart.smoke.spec.ts
│   │   ├── search.smoke.spec.ts
│   │   ├── auth.smoke.spec.ts
│   │   └── checkout.smoke.spec.ts
│   ├── pages/
│   │   ├── HomePage.ts
│   │   ├── ProductPage.ts
│   │   ├── CartPage.ts
│   │   ├── LoginPage.ts
│   │   └── CheckoutPage.ts
│   ├── test-data/
│   │   └── users.ts
│   ├── utils/
│   │   └── testData.ts
│   ├── playwright.config.ts
│   ├── package.json
│   └── README.md
```

3. Installation

```
mkdir fashion-ecommerce-playwright
cd fashion-ecommerce-playwright
npm init -y
npm install -D @playwright/test typescript ts-node
npx playwright install
npm init playwright@latest
```

Select TypeScript, use the tests folder, enable GitHub Actions if desired, and install browsers.

4. Playwright Configuration

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  reporter: [['html', { open: 'never' }]], ['list']],

  use: {
    baseURL: 'https://fashion-ecommerce-we-lunb.bolt.host',
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'retain-on-failure',
    actionTimeout: 10000,
    navigationTimeout: 30000,
    ignoreHTTPSErrors: true,
  },

  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

5. Page Object Model

HomePage.ts

```
import { Page, Locator, expect } from '@playwright/test';

export class HomePage {
  readonly page: Page;
  readonly searchBox: Locator;
  readonly cartButton: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.searchBox = page.getByRole('textbox', { name: /search/i });
    this.cartButton = page.getByRole('link', { name: /cart/i });
    this.loginButton = page.getByRole('link', { name: /login|sign in/i });
  }

  async navigate() {
    await this.page.goto('/');
  }

  async verifyHomePage() {
    await expect(this.page).toHaveTitle(/fashion/i);
  }

  async searchProduct(product: string) {
    await this.searchBox.fill(product);
    await this.searchBox.press('Enter');
  }

  async openCart() {
    await this.cartButton.click();
  }
}
```

ProductPage.ts

```
import { Page, Locator, expect } from '@playwright/test';

export class ProductPage {
  readonly page: Page;
  readonly products: Locator;
  readonly productName: Locator;
  readonly productPrice: Locator;
  readonly addToCartButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.products = page.locator('[data-testid="product-card"]');
    this.productName = page.locator('[data-testid="product-name"]');
    this.productPrice = page.locator('[data-testid="product-price"]');
    this.addToCartButton = page.getByRole('button', { name: /add to cart/i });
  }

  async openFirstProduct() {
    await this.products.first().click();
  }

  async verifyProductDetails() {
    await expect(this.productName).toBeVisible();
    await expect(this.productPrice).toBeVisible();
    await expect(this.addToCartButton).toBeVisible();
  }

  async addToCart() {
    await this.addToCartButton.click();
  }
}
```

CartPage.ts

```
import { Page, Locator, expect } from '@playwright/test';

export class CartPage {
  readonly page: Page;
  readonly cartItems: Locator;
  readonly checkoutButton: Locator;
  readonly removeButton: Locator;
```

```

constructor(page: Page) {
  this.page = page;
  this.cartItems = page.locator('[data-testid="cart-item"]');
  this.checkoutButton = page.getByRole('button', { name: /checkout/i });
  this.removeButton = page.getByRole('button', { name: /remove/i });
}

async verifyCart() {
  await expect(this.cartItems.first()).toBeVisible();
}

async checkout() {
  await this.checkoutButton.click();
}

async removeProduct() {
  await this.removeButton.first().click();
}
}

```

LoginPage.ts

```

import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly email: Locator;
  readonly password: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.email = page.getByRole('textbox', { name: /email/i });
    this.password = page.getByLabel(/password/i);
    this.loginButton = page.getByRole('button', { name: /login|sign in/i });
  }

  async login(email: string, password: string) {
    await this.email.fill(email);
    await this.password.fill(password);
    await this.loginButton.click();
  }
}

```

6. Smoke Test Examples

Home smoke tests

```
import { test, expect } from '@playwright/test';
import { HomePage } from '../../pages/HomePage';

test.describe('Fashion E-Commerce - Home Smoke Suite', () => {

  test('SMK-001 - Application should load successfully', async ({ page }) => {
    const homePage = new HomePage(page);
    await homePage.navigate();
    await expect(page).toHaveURL(/fashion-ecommerce/);
    await homePage.verifyHomePage();
  });

  test('SMK-002 - Home page should display navigation', async ({ page }) => {
    const homePage = new HomePage(page);
    await homePage.navigate();
    await expect(homePage.cartButton).toBeVisible();
  });

  test('SMK-012 - Search field should be available', async ({ page }) => {
    const homePage = new HomePage(page);
    await homePage.navigate();
    await expect(homePage.searchBox).toBeVisible();
  });
});
```

Product smoke tests

```
import { test, expect } from '@playwright/test';
import { HomePage } from '../../pages/HomePage';
import { ProductPage } from '../../pages/ProductPage';

test.describe('Product Smoke Suite', () => {

  test('SMK-003 - Product listing should be displayed', async ({ page }) => {
    const homePage = new HomePage(page);
    const productPage = new ProductPage(page);
    await homePage.navigate();
    await expect(productPage.products.first()).toBeVisible();
  });

  test('SMK-005 - User should be able to open product details', async ({ page }) => {
    const homePage = new HomePage(page);
    const productPage = new ProductPage(page);
    await homePage.navigate();
    await productPage.openFirstProduct();
    await productPage.verifyProductDetails();
  });

  test('SMK-007 - User should be able to add product to cart', async ({ page }) => {
    const homePage = new HomePage(page);
    const productPage = new ProductPage(page);
    await homePage.navigate();
    await productPage.openFirstProduct();
    await productPage.addToCart();
    await expect(page.getByText(/added to cart|cart/i).first()).toBeVisible();
  });
});
```

Cart smoke tests

```
import { test, expect } from '@playwright/test';
import { HomePage } from '../../pages/HomePage';
import { ProductPage } from '../../pages/ProductPage';
import { CartPage } from '../../pages/CartPage';

test.describe('Cart Smoke Suite', () => {

  test.beforeEach(async ({ page }) => {
    const homePage = new HomePage(page);
    const productPage = new ProductPage(page);
    await homePage.navigate();
    await productPage.openFirstProduct();
    await productPage.addToCart();
  });

  test('SMK-008 - Cart should open successfully', async ({ page }) => {
    const homePage = new HomePage(page);
```

```

    const cartPage = new CartPage(page);
    await homePage.openCart();
    await cartPage.verifyCart();
  });

  test('SMK-009 - Cart should contain selected product', async ({ page }) => {
    const homePage = new HomePage(page);
    const cartPage = new CartPage(page);
    await homePage.openCart();
    await expect(cartPage.cartItems.first()).toBeVisible();
  });

  test('SMK-011 - User should be able to remove product', async ({ page }) => {
    const homePage = new HomePage(page);
    const cartPage = new CartPage(page);
    await homePage.openCart();
    await cartPage.removeProduct();
  });
});

```

Search smoke test

```

test('SMK-013 - Valid product search should return results', async ({ page }) => {
  const homePage = new HomePage(page);

  await homePage.navigate();
  await homePage.searchProduct('shirt');

  await expect(page).toHaveURL(/search/i);
  await expect(
    page.locator('[data-testid="product-card"]').first()
  ).toBeVisible();
});

```

7. End-to-End Smoke Test

```
test(
  'SMK-E2E-001 - User should complete basic shopping journey',
  async ({ page }) => {

    await page.goto('/');

    await expect(page).toHaveURL(/fashion-ecommerce/);

    await page.locator('[data-testid="product-card"]').first().click();

    await expect(
      page.locator('[data-testid="product-name"]')
    ).toBeVisible();

    await page.getByRole('button', {
      name: /add to cart/i
    }).click();

    await page.getByRole('link', {
      name: /cart/i
    }).click();

    await expect(
      page.locator('[data-testid="cart-item"]')
    ).toBeVisible();

    await page.getByRole('button', {
      name: /checkout/i
    }).click();

    await expect(
      page.getByRole('heading', {
        name: /checkout/i
      })
    ).toBeVisible();
  }
);
```

8. Smoke Tagging and Execution

```
test('SMK-001 @smoke Application should load successfully', async ({ page }) => {
  // test
});
```

```
npx playwright test --grep @smoke
npx playwright test --project=smoke
npx playwright test --project=smoke --headed
npx playwright test --project=smoke --debug
npx playwright show-report
```

9. Recommended Automation IDs

Component	Recommended data-testid
Product card	product-card
Product name	product-name
Product price	product-price
Product image	product-image
Add to cart	add-to-cart
Cart item	cart-item
Cart quantity	cart-quantity
Remove product	remove-cart-item
Cart total	cart-total
Checkout	checkout-button
Email	login-email
Password	login-password

Component	Recommended data-testid
Login	login-button
Address	shipping-address
City	shipping-city
Pincode	shipping-pincode
Place order	place-order
Order number	order-number

10. Locator Strategy

Prefer Playwright's `getByRole()`, then `getByLabel()`, then `getByTestId()`. Avoid fragile CSS/XPath selectors such as `nth-child` chains whenever possible.

```
npx playwright codegen https://fashion-ecommerce-we-lunb.bolt.host/
```

Use Playwright Codegen locally to discover the application's actual selectors and then convert them into maintainable Page Object locators.

11. GitHub Actions Smoke Pipeline

```
name: Fashion E-Commerce Smoke Tests

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

jobs:
  smoke:
    timeout-minutes: 15
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright
        run: npx playwright install --with-deps chromium

      - name: Run Smoke Tests
        run: npm run smoke

      - name: Upload Playwright Report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: playwright-report/

      - name: Upload Test Results
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: test-results/
```

12. package.json Scripts

```
{
  "scripts": {
    "test": "playwright test",
    "smoke": "playwright test --project=smoke",
    "smoke:headed": "playwright test --project=smoke --headed",
    "smoke:debug": "playwright test --project=smoke --debug",
    "report": "playwright show-report"
  }
}
```

13. Recommended Automation Architecture

Fashion E-Commerce → Playwright + TypeScript → Page Objects + Test Data → Smoke Tests → Home / Product / Cart / Checkout / Order → HTML Report → GitHub Actions. Keep smoke tests business-critical and fast; move deeper validations into regression suites.

Important implementation note

The code in this document uses resilient role/text selectors and recommended data-testid contracts. Because the application's exact DOM was not available for reliable inspection in the generation environment, the selectors should be validated against the running application with Playwright Codegen/Inspector before execution.